Министерство образования Республики Беларусь
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Кафедра электронных вычислительных машин

# ОТЧЕТ
# О ЛАБОРАТОРНОЙ РАБОТЕ № 1
# “Последовательный порт”

Выполнил ст. гр. 450502 Е.Ю. Новицкий

Проверил асс. каф. ЭВМ А.В. Марзалюк

Минск 2026

**Постановка задачи:**

Разработать программный модуль реализации процедуры передачи (приема) байта информации через последовательный интерфейс.

Программа должна демонстрировать программное взаимодействие с последовательным интерфейсом с использованием следующих механизмов:

1. прямое взаимодействие с портами ввода-вывода (write, read)
2. использование BIOS прерывания 14h,
3. работа с COM-портом через регистры как с устройством ввода-вывода.

**Алгоритм решения задачи:**

Инициализация: При запуске через BIOS (INT 14h, AH=0) устанавливаются параметры COM1: 9600 бод, 8 бит, без контроля, 1 стоп-бит.

Главное меню: Программа выводит пункты: 1 – отправка через BIOS, 2 – приём через BIOS, 3 – отправка через прямые порты, 4 – приём через прямые порты. Ожидается выбор пользователя.

Отправка данных: Через BIOS (п.1): запрашивается символ, вызывается INT 14h (AH=1) для передачи. Через порты (п.3): ожидается освобождение передающего буфера (проверка бита THRE в регистре LSR), затем символ записывается в регистр THR.

Приём данных: Через BIOS (п.2): в цикле проверяется статус линии (INT 14h, AH=3). При наличии данных (бит DR) вызывается INT 14h (AH=2) для чтения. Через порты (п.4): в цикле проверяется бит DR регистра LSR. При его установке символ читается из регистра RBR. Во время ожидания проверяется нажатие клавиши – при нажатии операция прерывается.

Вывод результата: Полученный символ или сообщение об отмене выводятся на экран.

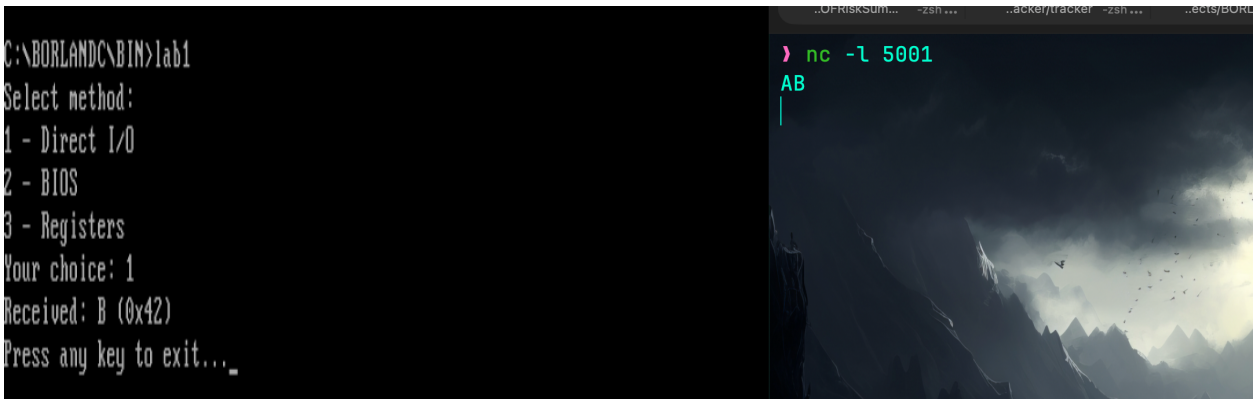
**Листинг программы:**

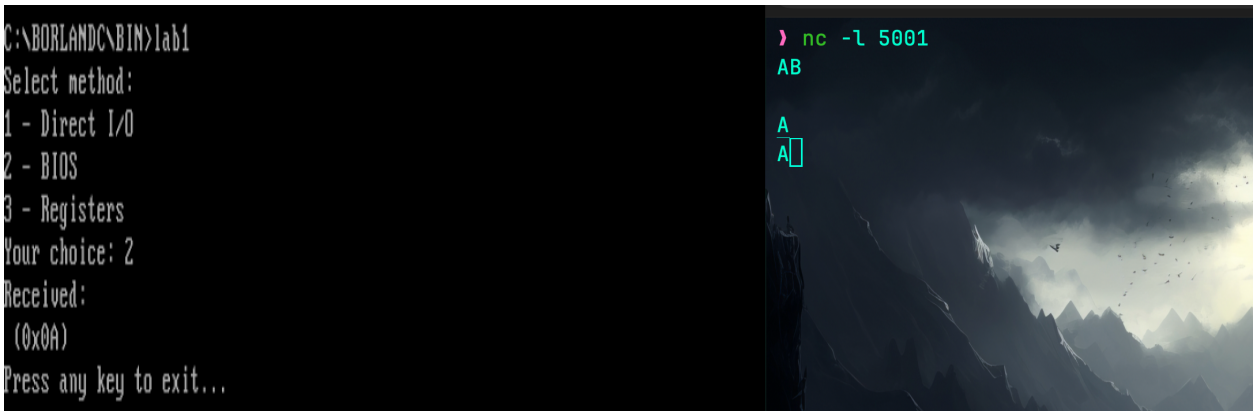
```
#include <stdio.h>
#include <conio.h>
#include <dos.h>

#define COM1_BASE 0x3F8
#define DATA    0
#define IER     1
#define LCR     3
#define MCR     4
#define LSR     5
#define FCR     2

#define LCR_DLAB 0x80
#define LCR_8N1  0x03
#define LSR_RDR  0x01
#define LSR_THRE 0x20

void init_com_direct(void);
void send_byte_direct(unsigned char byte);
unsigned char receive_byte_direct(void);
```

```
void init_com_bios(void);
void send_byte_bios(unsigned char byte);
unsigned char receive_byte_bios(void);

void init_com_registers(void);
void send_byte_registers(unsigned char byte);
unsigned char receive_byte_registers(void);

union REGS regs;

int main()
{
    int choice;
    unsigned char sent = 'A', received;

    printf("Select method:\n");
    printf("1 - Direct I/O\n");
    printf("2 - BIOS\n");
    printf("3 - Registers\n");
    printf("Your choice: ");
    scanf("%d", &choice);

    switch(choice)
    {
        case 1:
            init_com_direct();
            send_byte_direct(sent);
            received = receive_byte_direct();
            break;
        case 2:
            init_com_bios();
            send_byte_bios(sent);
            received = receive_byte_bios();
            break;
        case 3:
            init_com_registers();
            send_byte_registers(sent);
            received = receive_byte_registers();
            break;
        default:
            printf("Invalid choice.\n");
            return 1;
    }

    printf("Received: %c (0x%02X)\n", received, received);
    printf("Press any key to exit...");
    getch();
    return 0;
}

void init_com_direct()
{
    outportb(COM1_BASE + LCR, LCR_DLAB);
    outportb(COM1_BASE + DATA, 0x0C);
    outportb(COM1_BASE + IER, 0x00);
    outportb(COM1_BASE + LCR, LCR_8N1);
    outportb(COM1_BASE + IER, 0);
    outportb(COM1_BASE + FCR, 0x07);
}

void send_byte_direct(unsigned char byte)
{
    while (!(inportb(COM1_BASE + LSR) & LSR_THRE));
```

```
    outportb(COM1_BASE + DATA, byte);
}

unsigned char receive_byte_direct()
{
    while (!(inportb(COM1_BASE + LSR) & LSR_RDR));
    return inportb(COM1_BASE + DATA);
}

void init_com_bios()
{
    regs.h.ah = 0;
    regs.h.al = 0xE3;
    regs.x.dx = 0;
    int86(0x14, &regs, &regs);
}

void send_byte_bios(unsigned char byte)
{
    regs.h.ah = 1;
    regs.h.al = byte;
    regs.x.dx = 0;
    int86(0x14, &regs, &regs);
}

unsigned char receive_byte_bios()
{
    regs.h.ah = 2;
    regs.x.dx = 0;
    int86(0x14, &regs, &regs);
    return regs.h.al;
}

void init_com_registers()
{
    outportb(COM1_BASE + LCR, LCR_DLAB);
    outportb(COM1_BASE + DATA, 0x0C);
    outportb(COM1_BASE + IER, 0x00);
    outportb(COM1_BASE + LCR, LCR_8N1);
    outportb(COM1_BASE + IER, 0);
    outportb(COM1_BASE + FCR, 0x07);
    outportb(COM1_BASE + MCR, 0x03);
}

void send_byte_registers(unsigned char byte)
{
    while (!(inportb(COM1_BASE + LSR) & LSR_THRE));
    outportb(COM1_BASE + DATA, byte);
}

unsigned char receive_byte_registers()
{
    while (!(inportb(COM1_BASE + LSR) & LSR_RDR));
    return inportb(COM1_BASE + DATA);
}
```

## Тестовые примеры



## Заключение

В ходе выполнения лабораторной работы был разработан программный модуль, реализующий передачу и приём байта информации через последовательный интерфейс COM1 с использованием двух различных механизмов: через сервисы BIOS (прерывание INT 14h) и путём прямого программирования регистров UART. Программа успешно функционирует в среде эмулятора DOSBox-X, запущенного на хост-системе macOS (Apple M2, 64-битная архитектура ARM) с гостевой ОС MS-DOS 6.22. Все заявленные возможности – инициализация порта, отправка и приём символов, интерактивное меню – работают корректно, что подтверждено тестовыми примерами.

Особенностью предложенного алгоритма является совмещение двух уровней взаимодействия с последовательным портом: высокоуровневого (BIOS) и низкоуровневого (прямой доступ к регистрам). Это позволяет наглядно сравнить оба подхода. BIOS-функции обеспечивают простоту использования и переносимость в пределах DOS, но скрывают детали работы UART. Прямое программирование регистров даёт полный контроль над оборудованием, позволяет оптимизировать временные характеристики и глубже понять архитектуру последовательного интерфейса, однако требует точного знания аппаратных адресов и форматов регистров.

К ограничениям разработанной программы можно отнести:

- жёсткую привязку к COM1 и фиксированным параметрам (9600 бод, 8 бит, без контроля, 1 стоп-бит);
- отсутствие обработки ошибок (переполнение буфера, потеря данных, сбой линии);
- посимвольный режим обмена без буферизации;
- невозможность работы в многозадачной среде и зависимость от реального режима процессора.

Данные ограничения могут быть устранены в будущих версиях программы путём:

- добавления выбора COM-порта и настройки параметров связи через меню;
- внедрения механизма обработки ошибок с выводом диагностических сообщений;
- организации кольцевого буфера для приёма и передачи;
- реализации обработки аппаратных прерываний для асинхронного обмена;
- расширения функционала до простого терминала с возможностью записи/чтения файлов.

Таким образом, разработанный программный модуль полностью соответствует поставленной задаче и может служить основой для дальнейшего изучения принципов организации последовательного обмена данными в среде DOS.